Highlights

RoomRecon: Walkable Indoor Reconstruction from Smartphone Captures with Depth-Guided Gaussian Splatting, and an Evaluation of Walk-Collision Accuracy

Aswin M, Sejal Satheesh

- An iPhone room capture becomes a photoreal Gaussian splat you can walk through.
- Built by reusing maintained tools; our code is the glue and the evaluation.
- Collision from a depth-guided mesh fits a laser scan far better than from splats.
- Two other novelty attempts failed their pre-agreed tests; both are reported.

RoomRecon: Walkable Indoor Reconstruction from Smartphone Captures with Depth-Guided Gaussian Splatting, and an Evaluation of Walk-Collision Accuracy

Aswin M[a,1,*], Sejal Satheesh[a,2]

[a] *Indian Institute of Information Technology, Kottayam, India*

Abstract

A photoreal 3D copy of a room that a person can walk through needs a model that looks right from anywhere in the room and collision that stops the walker at the real walls. Gaussian splatting provides the first, but plain walls defeat camera poses estimated from images, and the walk collision that viewers build from a splat has not been checked against the real room. We present RoomRecon, which turns an iPhone capture (ARKit poses and LiDAR depth) into a first-person walkable model by reusing maintained tools (nerfstudio with DN-Splatter, splat-transform and the SuperSplat Viewer), and adds the glue they lack and an evaluation of walk collision against a laser scan. On the MuSHRoom `vr_room` scene, DN-Splatter reached 26.3 dB PSNR on held-out frames with a mesh 1.8 cm from the laser scan (plain splats: 5.9 cm); our own room reached 22.2 dB, and both rooms were walkable on

*Corresponding author
[1]Roll No. 2023BCS0012
[2]Roll No. 2023BCS0048

a laptop GPU. Collision built from DN-Splatter's mesh matched the scan's walkable area with an IoU of 0.81 and missed 1.7% of the walls, against 0.58 and 17.8% for the default collision built from the splat; Polycam's mesh scored 0.79 and 1.8%. The mesh met three of our four pre-agreed criteria, although a free walk did not reveal the difference. Two other novelty attempts failed their pre-agreed tests and are reported as negative results.
Keywords: Gaussian splatting, Indoor reconstruction, Smartphone capture, LiDAR, Walk collision, Virtual walkthrough

1. Introduction

A 3D copy of a real room that looks real and can be explored on foot is useful for visiting a place remotely, for planning changes to a space and for testing robots that move indoors [1, 2]. Neural radiance fields [3] and 3D Gaussian splatting (3DGS) [4] reconstruct such copies from photographs, and 3DGS renders them in real time. Capture has become cheap: an iPhone records each frame with its own camera pose from ARKit, and the Pro models add a LiDAR depth map.

Getting from a phone capture to a room one can walk in still has two weak points. First, rooms are dominated by plain walls, ceilings and floors. Camera poses estimated from the images fail on such surfaces, and the trained model has little to constrain its geometry; DN-Splatter [5] and later methods [6, 7, 8] add depth, normal or planar priors for this reason. Second, walking needs collision that stops the walker at the walls. The SuperSplat Viewer's

walk mode [9, 10] builds it automatically from the splat with splat-transform [11], but we found no measurement of how closely this collision follows the real walls.

Our first attempt (v1, August–September 2026) estimated poses from images and failed on a plain-walled bedroom (Section 3). We restarted as RoomRecon v2 around one rule: reuse maintained tools and write only the glue between them. The phone supplies poses and depth, DN-Splatter on nerfstudio [12] trains the model on a shared GPU, and splat-transform and the SuperSplat Viewer provide collision and walking on a laptop (Figure 1).



Figure 1 First-person walk in the SuperSplat Viewer on a laptop (RTX 3050, 6 GB), frames from our screen recordings. Left: MuSHRoom `vr_room` (0.91 M Gaussians). Right: our own room, captured on an iPhone 13 Pro (0.75 M Gaussians). The lower-left overlay is the viewer's frame-time counter.

The salient contributions of this work are:

1. **A working pipeline from an iPhone capture to a walkable Gaussian splat**, assembled from existing tools plus our glue (training settings for a shared GPU, automatic alignment of up direction, floor, scale and starting point, offline demo bundles), shown on a public room with a laser scan (MuSHRoom `vr_room` [13]) and on our own room.

2. **A measurement of walk-collision accuracy against a laser scan**, comparing collision built from DN-Splatter's mesh with the default built from the splat, with raw phone LiDAR and with Polycam's mesh, under criteria fixed before the run.
3. **An account of what we explored and learned**, including the v1 failures that shaped v2 and two novelty attempts that failed their pre-agreed tests and are reported as negative results.

The organisation of the report is as follows. Section 2 discusses related work and its limitations. Section 3 describes how we explored the problem and what we learned. Section 4 presents the pipeline and the collision evaluation. Section 5 gives the datasets, setup and results, including a comparison with existing implementations. Section 6 concludes with two directions for future work.

2. Related works

This section discusses the work our pipeline builds on and is measured against, grouped by the question each line of work answers. For each work we give the problem, the method and the outcome reported by its authors.

2.1. Radiance fields and splatting toolkits

NeRF [3] represents a scene as a neural field rendered by volume rendering; 3DGS [4] replaces it with millions of anisotropic Gaussians rasterised in real time. Nerfstudio [12] is a modular framework for such methods, with

data importers, a viewer and export tools, and its rasteriser library gsplat [14] reports up to 10% less training time and 4× less memory than the original 3DGS code. 2D Gaussian splatting [15] improves the poor surfaces of 3DGS by collapsing the Gaussians into oriented disks with depth-distortion and normal-consistency terms, and reports detailed, noise-free geometry. 3DGS-MCMC [16], used in our v1, recasts densification as Markov chain Monte Carlo sampling.

2.2. Gaussian splatting for indoor rooms

DN-Splatter [5] targets textureless indoor scenes: it supervises the Gaussians with sensor depth through a loss weighted by image gradients, adds a smoothness term and monocular normals, and extracts a mesh directly from the result. AGS-Mesh [6] notes that phone depth is coarse and monocular normals are inconsistent across views, filters these priors by their consistency during training and adds a scale-aware meshing step, improving meshes and novel views on indoor room datasets. GaussianRoom [17] couples a neural signed distance field with 3DGS to guide densification and pruning, with normal and edge priors for textureless areas, and reports state-of-the-art results on ScanNet and ScanNet++. IndoorGS [7] lifts 2D line segments to 3D and derives plane cues for textureless regions to drive initialisation and density control, reporting more accurate geometry and rendering. PlanarGS [8] finds planar regions with a vision-language segmentation model and supervises the Gaussians with planar and depth/normal terms, outperforming earlier meth-

ods on indoor benchmarks. LighthouseGS [18] handles panorama-style phone captures by using the phone's poses and monocular depth as rough priors, initialising on planes and correcting drift and auto-exposure. PocketGS [19] trains 3DGS on the phone itself within minutes. All of these report image quality and surface accuracy; none asks whether a person can walk in the result.

2.3. Camera poses from images alone

COLMAP [20] recovers poses by matching local features; learned features such as SuperPoint [21] with LightGlue [22] in hloc [23], and visual SLAM such as ORB-SLAM3 [24], rely on the same principle. MASt3R-SfM [25] builds structure from motion on a 3D foundation model and aligns its local reconstructions at linear cost; VGGT [26] predicts cameras, depth and point maps from up to hundreds of views in one forward pass, in under a second. These newer models depend less on texture than the methods that failed in our v1, but we did not test them; taking poses from the phone removed the problem for this project.

2.4. Walking and navigating in Gaussian splats

The SuperSplat Viewer [10] offers a first-person walk mode whose collision is a voxel volume generated from the splat by splat-transform [11, 9]. Splat-Nav [27] plans safe robot trajectories directly in a splat map using collision constraints derived from the Gaussians, tested in 126 hardware flights.

NavGSim [2] is a splat-based simulator for multi-room navigation that extracts navigable areas by slicing the Gaussians. The Neverwhere benchmark [1] turns more than sixty splat reconstructions into closed-loop tests for visual legged locomotion, and LucidSim [28] trains a visual parkour policy from generated images; in both, physics runs on separate collision geometry while the images come from the splat or a generator. None of these reports how accurately its collision follows the real walls.

2.5. Capture guidance and view selection

Kopanas and Drettakis [29] place cameras for free-viewpoint navigation in rooms using observation frequency and angular uniformity. FisherRF [30] picks the next view by expected information gain from the Fisher information of the model's parameters, while NeRF Director [31] shows that uniform coverage alone, without any uncertainty estimate, selects good training views. COVER [32] shows that an approximation of the Fisher information gain reduces to covering geometry that past cameras saw poorly, and AGILE-GS [33] cuts next-best-view selection time by one to two orders of magnitude. For people capturing with phones, Zhang et al. [34] guide object captures live by spherical coverage and beat free capture with fewer images; Yasunaga et al. [35] show in augmented reality where a quick reconstruction is wrong so that the user adds views there; LiMRSF [36] overlays a LiDAR SLAM point cloud on the real room in a mixed-reality headset to reveal blind spots and double walls while scanning. MUGSQA [37] collects human quality ratings of

splats reconstructed from inputs of varying number, resolution and viewing distance.

2.6. Limitations of existing work

Limitations of the existing methods, as they affect a walkable room, are:

- Indoor splatting methods are evaluated on image and mesh quality only; walkability is not tested.
- Walk collision built from a splat is the default in a widely used viewer, but its accuracy against a ground-truth scan is not published; robotics work pairs splats with separate collision geometry without measuring the difference.
- Pose estimation from images alone still struggles in plain-walled rooms (our v1; the newest feed-forward models were not tested).
- Capture feedback is live, object-scale, or needs special hardware or a trained model; none checks an exported room capture before training.
- The path from a phone to a walkable room spans separate tools with different coordinate conventions and defaults that do not suit a shared GPU.

We address the first, second and last points directly (Sections 4 and 5.4). The fourth was our first novelty attempt, which failed its own test (Section 5.5); the third is future work.

3. What we explored and learned

The pipeline of Section 4 is the result of two attempts, several failures and a few measurements that changed our plans (Figure 2). This section follows them in order, because most of what we learned came from them.

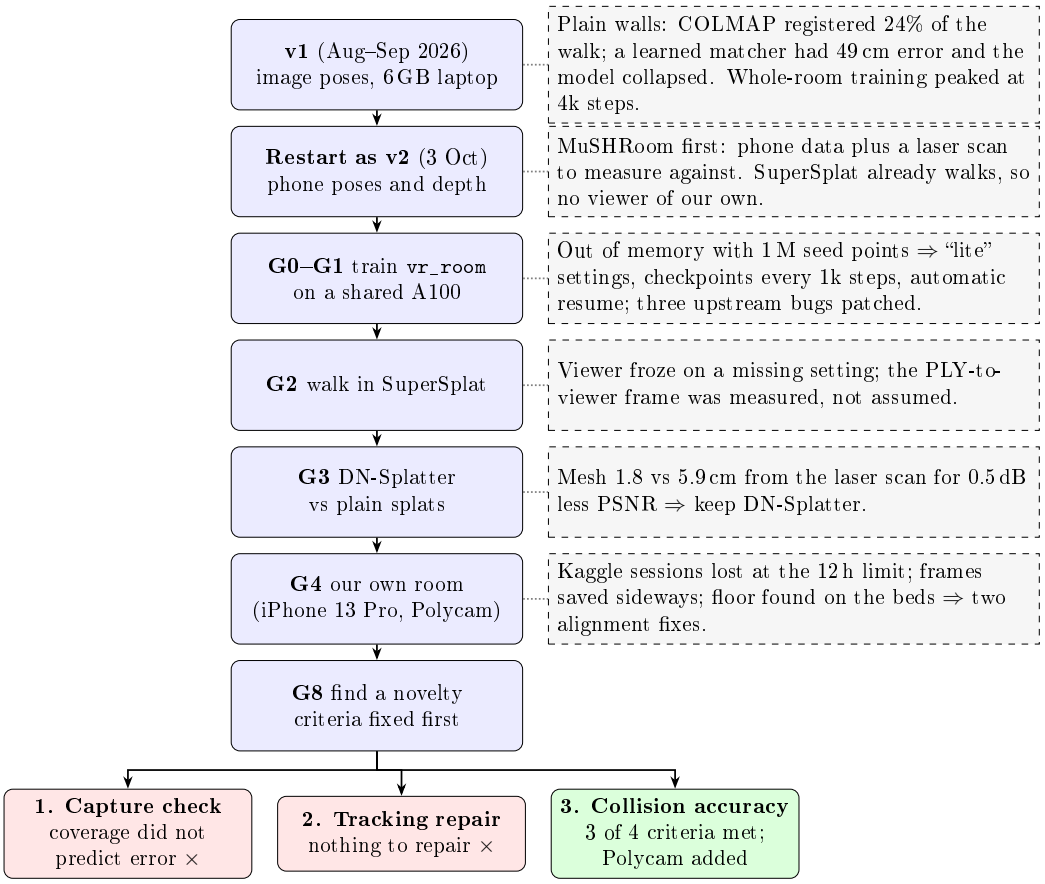


Figure 2 How the project developed. Blue: goals worked through in order; dashed: what each step taught us; red and green: the three novelty attempts and whether they met the criteria fixed before running them.

3.1. v1: poses from images on a laptop

Our first version aimed to reconstruct a room from RGB images alone and to train on a laptop (RTX 3050, 6 GB). The test room was a plain-walled ARKitScenes [38] bedroom (scene 41254269, a 95 s iPad walk). Table 1 compares the pose sources we tried against the iPad's own ARKit poses. COLMAP [20] was accurate but registered only a quarter of the walk; SuperPoint [21] and LightGlue [22] in hloc [23] registered every frame with 49 cm error, and 3DGS trained on those poses rendered every view as flat colour. A textured utility room, by contrast, tracked cleanly (COLMAP: all frames, 1.6 cm). Image-only poses failed specifically on plain walls, and accuracy mattered more than coverage.

Table 1 v1 pose sources on the plain-walled ARKitScenes bedroom 41254269 (laptop). Error is measured against the iPad's ARKit poses.

Pose source	**Walk covered**	**Error**	**3DGS result**
COLMAP, SIFT	24%	1.2 cm	sharp, a quarter of the room
SuperPoint + LightGlue	100%	49 cm	total collapse
ORB-SLAM3 [24], mono	7–93%	6–52 cm	not trained (4–6 maps)
ARKit device poses	100%	reference	see Figure 3

Even with ARKit poses, whole-room training did not hold: a 22 s window of the walk reached 30.3 dB on held-out frames, but the full walk peaked at 22.8 dB after 4,000 steps and fell to 13.7 dB by 15,000 (Figure 3, left). Switching densification from MCMC [16] to the default only slowed the decline, and we never found the cause. Only evaluating every 1,000 steps revealed the early peak; the final number alone said "failure". v1 also showed that its browser viewer, GaussianSplats3D [39], dropped to 8 frames per second (95th

percentile) at a million Gaussians, and that a 32-photo phone capture made with NeRFCapture [40] trained to only 16.0 dB, full of streaks and floaters.

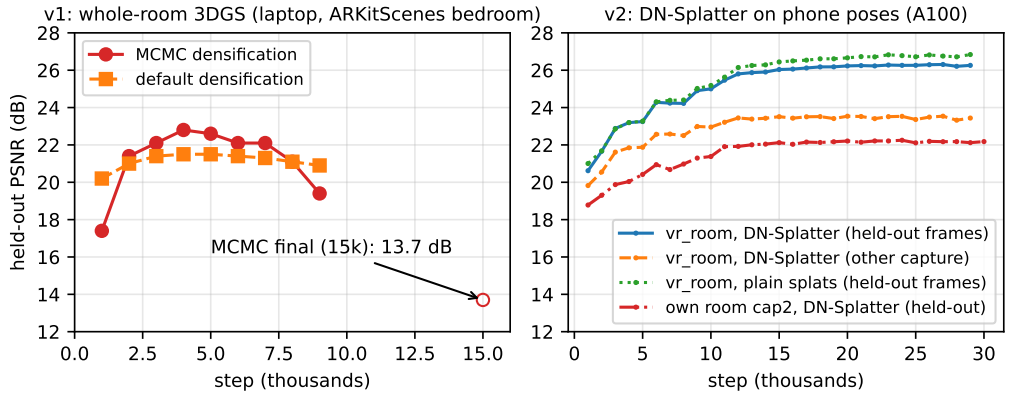


Figure 3 Held-out PSNR during training, evaluated every 1,000 steps. Left: v1, the full bedroom walk with ARKit poses on the laptop (from our v1 log; after 9k only the final value at 15k was recorded). Right: v2 on the A100: DN-Splatter on `vr_room` (held-out frames of the training walk and frames of a separate walk), plain splats on the same data, and our own room. No v2 run collapsed.

3.2. Restarting as v2

We kept the goal and changed the approach. The phone tracks its own pose by combining the camera with its inertial sensors, and those poses had trained a usable model where image-only poses had not, so v2 takes poses (and LiDAR depth) from the phone. DN-Splatter was chosen because it targets plain indoor surfaces and ships a loader for MuSHRoom, whose rooms come with iPhone data and a laser scan. SuperSplat already offered first-person walking with collision, so our plan to build a viewer in Unity was dropped. Training moved to a campus A100, and the work was organised as

goals in priority order, each with exit criteria, so that unfinished goals could be cut from the end.

3.3. Research code on a shared GPU

Another user held 33–35 GB of the A100's 40 GB. DN-Splatter seeds one million Gaussians by default, and our first full run ran out of memory at about step 1,000. We changed as little as possible: 300k seed points, splitting stopped at step 10,000 instead of 15,000, and a wrapper that resumes from the newest checkpoint after a crash; a deliberate kill-and-resume test continued at step 1,001. Three upstream bugs had to be patched (an evaluation key error, a crash on resuming with an empty optimiser state, and resumed runs repeating their full step count), and the environment needed pins against conflicting dependency versions. Research code assumes a dedicated GPU and one uninterrupted run; checkpoints, regular evaluation and automatic resume are what made our runs finish.

3.4. From a trained model to a walkable room

A trained splat has no notion of up, floor, size or starting point. The viewer loads a PLY rotated by 180° about z, and its Euler-angle convention is undocumented; we measured both rather than assume them. The first walk test failed because our settings file lacked a block the viewer's newer schema requires. Our own room then broke two assumptions of our alignment. Polycam had saved every frame sideways, so the cameras' mean up vector averaged to almost nothing; up is now taken from the camera's $-x$

axis when $+y$ disagrees. The floor had been taken as the densest level below the cameras, which here was the tops of the beds, and the room came out 1.68 times too large; the floor is now the lowest dense level (Algorithm 1). Against MuSHRoom's laser scan, the old rule had put the `vr_room` floor 13 cm too high and the new one is within about 1.5 cm. The released `vr_room` demo predates this fix and was deliberately kept as walk-tested.

3.5. Capturing our own room

Sejal captured a bedroom three times on an iPhone 13 Pro with Polycam's raw-data export [41]. The first capture (32 frames) was too sparse, as v1 had predicted; the second (257 frames, one tracking segment) covered the room and was used; the third (491 frames) had three tracking segments with pose jumps of about 2 m. While the A100 was full we tried Kaggle's free GPUs, but both sessions ran to Kaggle's 12-hour limit without saving output, most likely because our memory sampler outlived the training script (now stopped explicitly). Back on the A100, the run crashed once at step 10,400, resumed by itself from step 10,000 and finished.

3.6. Looking for a novelty

For each novelty idea we searched the literature first and fixed pass/fail criteria in writing before running anything. A capture check before training (Section 4.6) failed its validation; a repair of `cap3`'s broken tracking found nothing to repair; the accuracy of walk collision met three of four criteria and is the result we lead with (Section 5).

The lessons that shaped the final design:

- take poses from the phone, since image-only poses failed on plain walls;
- evaluate every 1,000 steps and look at renders, since a final number alone hid a collapse;
- measure coordinate conventions and check alignment against a scan, since two of our assumptions were wrong on real data;
- assume the GPU will run out: checkpoint, resume and keep evidence off the machine;
- fix success criteria before the run and report the outcome either way.

4. Proposed method

RoomRecon turns a phone capture into a Gaussian splat that can be walked through with collision, and measures how accurate that collision is. Almost every stage is an existing program; our code connects the stages, adapts their defaults and adds the alignment and evaluation steps that do not exist elsewhere (Figure 4).

4.1. Capture and import

Own rooms are captured with Polycam's raw-data export [41] on an iPhone with LiDAR: images, ARKit poses and intrinsics, a depth and confidence map per frame, and per-frame tracking data. nerfstudio's `ns-process-data`

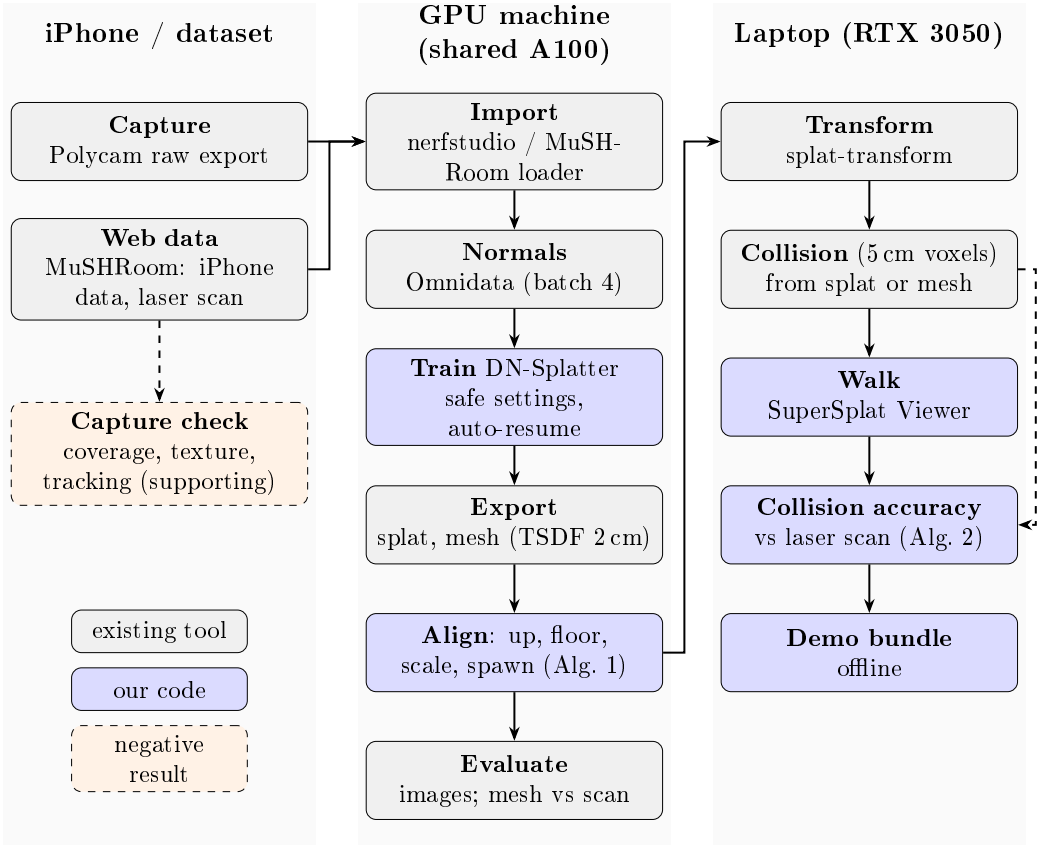


Figure 4 Architecture of RoomRecon. Grey: existing tools, used as they are or with small patches; blue: boxes containing our code (configuration, alignment, evaluation, packaging); orange: the supporting capture check, which failed its validation. Heavy work runs on the shared GPU machine; the laptop builds collision, shows the walk and runs the collision evaluation.

`polycam --use-depth` converts it into a dataset; for MuSHRoom, DN-Splatter's own loader is used. Normals for the normal prior come from Omnidata [42] through DN-Splatter's script, run in batches of four because the default batch does not fit beside the other GPU user. The initial Gaussians are 300,000 points back-projected from the LiDAR depth.

4.2. Training with settings that suit a shared GPU

DN-Splatter [5] adds three terms to the photometric loss of 3DGS: a depth loss against the LiDAR depth, weighted by image gradients so that edges are trusted less; a depth-smoothness term; and a loss against the monocular normals. We train for 30,000 steps with the changes in Table 2. Besides memory, the pose setting matters: nerfstudio re-centres and re-scales poses by default, which would make the exports non-metric, while the phone's poses are already metric.

Table 2 Training settings changed from the DN-Splatter / nerfstudio defaults. The two "lite" changes may cost quality; that cost was not measured.

Setting	**Default**	**Ours**	**Why**
Seed points (lite)	1,000,000	300,000	out of memory with 5–7 GB free
Stop splitting (lite)	step 15,000	step 10,000	caps the peak number of Gaussians
Checkpoints	only at the end	every 1k, keep all	a crash must not cost the run
Evaluation	effectively off	every 1k, all test images	a final-only metric hid a collapse
Pose processing	re-centre, re-scale	none (own data)	keep metric phone poses
Image cache	GPU	CPU	GPU memory
After a crash	manual restart	automatic resume	shared GPU

4.3. Export and scene alignment

The model is exported as a standard 3DGS PLY file and as a mesh, extracted by DN-Splatter with Open3D's TSDF fusion [43] at 2 cm voxels. Neither file says which way is up, where the floor is, how large the room is or

where to start. Algorithm 1 computes these from the cameras and the mesh, using two facts about handheld capture: the phone is held roughly upright, so the cameras' mean up vector gives gravity; and it is held near eye height, so the median camera height above the floor gives the scale. A scale near 1 confirms that the phone's poses were metric (1.038 for both of our rooms).

Algorithm 1 Scene alignment (`scene.json`)

procedure SceneFrame(camera-to-world poses $\{T_i\}$, mesh vertices V)
  $u \leftarrow$ mean of the cameras' $+y$ axes; **if** $\|u\| < 0.5$ **then** $u \leftarrow$ mean of $-x$ axes
  if $\|u\| < 0.5$ **then fail** "cannot estimate up"
  rotate camera centres c_i and V by R, which takes $u/\|u\|$ to $+y$
  $H \leftarrow$ histogram (2 cm bins) of vertex heights below the lowest camera
  $y_f \leftarrow$ centre of the lowest local peak of H holding $\geq 0.3 \max H$ ▷ not the bed tops
  $s \leftarrow 1.45\,\text{m} \,/\, \text{median}_i(c_{i,y} - y_f)$ ▷ eye height
  $k \leftarrow \arg\min_i \|c_i - \text{median}_j\, c_j\|$ ▷ a camera position is free space
  spawn $\leftarrow s\,c_k$ at height $s\,y_f + 1.45\,\text{m}$, facing camera k's horizontal view
  return R (as Euler angles in the viewer's frame), s, floor $s\,y_f$, spawn
end procedure

4.4. Collision and walking

On the laptop, splat-transform [11] applies the rotation and scale and builds the collision: it keeps the cluster of Gaussians connected to the spawn, voxelises it at 5 cm (solid above an opacity of 0.1), fills the space outside the room, carves the space reachable from the spawn by a capsule 1.6 m tall with a 0.2 m radius, and writes a collision mesh. The SuperSplat Viewer [10] 1.37.0 loads the splat, the collision and a settings file that places the start

camera at the spawn, and its walk mode gives first-person movement. We patched only its interface size. For a demo independent of the network and the GPU machine, a script packs the viewer and the room into one folder.

4.5. Evaluating walk-collision accuracy

Our main measurement compares collision from DN-Splatter's mesh with the default collision from the splat, on `vr_room`, which has a laser scan. Five sources go through exactly the demo's splat-transform pipeline, with the same rotation, scale and spawn: **S**, the trained splat (the default, used in our demo); **M**, points sampled every centimetre on DN-Splatter's mesh; **L**, the raw phone LiDAR (depth maps back-projected into 3 cm voxels, no training); **P**, Polycam's own mesh of the same capture, shipped with MuSHRoom and aligned to the scan by ICP from 24 starting orientations (a best case for Polycam); and **GT**, points on the laser scan, moved into the model's frame with MuSHRoom's alignment. M, L, P and GT are written as small opaque splats (2 cm) so that splat-transform treats them like a trained splat.

Each collision mesh is reduced to what matters for a walking person. Let O_X be the 5 cm floor-plan cells that contain collision surface between 0.2 and 1.8 m above the floor, W_X the free cells reachable from the spawn by four-connected steps, $d(c, O)$ the distance from cell c to the nearest cell of O, and $\mathcal{S}$ the GT-occupied cells that the phone's LiDAR also saw (within

10 cm). The measures are

$$\mathrm{IoU}_X = \frac{|W_X \cap W_{GT}|}{|W_X \cup W_{GT}|}, \qquad \mathrm{Phantom}_X = A\{c \in O_X \cap W_{GT} : d(c, O_{GT}) > 10\,\mathrm{cm}\}, \tag{1}$$

$$\mathrm{Missed}_X = \frac{|\{g \in \mathcal{S} : d(g, O_X) > 10\,\mathrm{cm}\}|}{|\mathcal{S}|}, \qquad \mathrm{Offset}_X = \{d(c, O_{GT}) : c \in O_X,\ d(c, O_{GT}) \le 50\,\mathrm{cm}\}, \tag{2}$$

where A is area and the offset is summarised by its median and 95th percentile. A phantom obstacle blocks a place where the real room is free; a missed wall lets the walker through a real wall.

Algorithm 2 Walk-collision accuracy against a laser scan

1: **procedure** CollisionAccuracy(collision meshes C_X, scan and LiDAR points, `scene.json`)
2: align the points with `scene.json`; $y_f \leftarrow$ floor of the scan (Alg. 1, line 6)
3: grid $\leftarrow$ 5 cm cells over the scan's floor plan plus a 0.5 m margin
4: **for each** source $X \in \{S, M, L, P, GT\}$ **do**
5: sample C_X every 1 cm; $O_X \leftarrow$ cells with samples at $0.2 \le y - y_f \le 1.8$ m
6: $W_X \leftarrow$ flood fill from the spawn cell through cells not in O_X
7: **end for**
8: compute Eqs. (1)–(2) for each X against GT; **pass** if M beats S on P1–P4
9: **end procedure**

The criteria, fixed before any run, were: P1, $\mathrm{IoU}_M \ge \mathrm{IoU}_S + 0.02$; P2, $\mathrm{Phantom}_M \le \mathrm{Phantom}_S$; P3, $\mathrm{Missed}_M \le \mathrm{Missed}_S + 2$ percentage points; P4, median $\mathrm{Offset}_M \le$ median Offset_S.

4.6. Capture check (supporting)

The capture check runs right after a capture, before training, on what the phone recorded. It merges every frame's LiDAR depth (high-confidence pixels) into 3 cm surface voxels, finds the frames that see each surface point by comparing its projection with each frame's own depth (so hidden walls do not count), and labels each point **under-covered** if fewer than 5 frames see it (the overlap that Pix4D's photogrammetry quality reports treat as sufficient) or its viewing directions fill fewer than 3 of 64 direction bins (COVER's angular coverage [32], moved before training); **textureless** if the median image gradient around it is below a threshold tuned on `vr_room` only; and **broken tracking** if every frame that sees it is flagged by a tracking-segment change, limited world mapping, a weakly connected frame, or a jump faster than 1.5 m/s or 180°/s. It outputs a verdict (OK, recapture, broken tracking), a top-down map and a coloured 3D map for the viewer.

5. Datasets and experimental results analysis

5.1. Datasets

Table 3 lists the data. The public room is `vr_room` from MuSHRoom [13] (CC BY 4.0), a room with plain concrete walls, chosen because it has the same kind of data as our phones (iPhone images, ARKit poses, LiDAR depth) plus a laser-scanned reference mesh. Its long capture trains the model with every tenth frame held out (the "within" test); its separate short capture, from viewpoints the training walk did not take, is the harder "with" test.

Our own room is a bedroom captured three times by Sejal Satheesh on an iPhone 13 Pro; every eighth frame of the used capture is held out.

Table 3 Datasets. Held-out frames are never used for training.

Room / sequence	**Source, device**	**Frames**	**Used for**
`vr_room` long capture	MuSHRoom, iPhone	422 (42 held out)	training, "within" test
`vr_room` short capture	MuSHRoom, iPhone	186	"with" test
`vr_room` laser scan	MuSHRoom reference	–	mesh and collision reference
Own room `cap1`	iPhone 13 Pro, Polycam	32	capture check (too sparse)
Own room `cap2`	iPhone 13 Pro, Polycam	257 (33 held out)	training, walk, capture check
Own room `cap3`	iPhone 13 Pro, Polycam	491 (3 segments)	capture check, tracking repair
ARKitScenes 41254269	iPad Pro	95 s walk	v1 only

Experimental setup. Training, export and mesh evaluation ran on a shared GPU machine: an NVIDIA A100-PCIE-40GB (MIG 7g.40gb) of which another user held 33–35 GB, with 96 CPU cores and 188 GB RAM, in an Ubuntu 20.04 container. The environment was built with pixi [44]: Python 3.10, PyTorch 2.2.2 with CUDA 11.8, nerfstudio 1.1.3, gsplat 1.0.0 and DN-Splatter at commit 97588b4 [45]. Each room was trained for 30,000 steps (Table 2) and evaluated on all held-out images every 1,000 steps. Mesh accuracy follows DN-Splatter's MuSHRoom protocol: the mesh is aligned to the scan with the dataset's ICP transform, both are culled to what the capture saw, and accuracy (mesh to scan), completeness (scan to mesh) and F-score at 5 cm are reported. "Plain splats" is the same model with every

prior switched off and 3D instead of flattened Gaussians, trained with the same data and settings. Collision, walking and the collision evaluation ran on the laptop (Windows 11, RTX 3050 Laptop GPU with 6 GB, Chrome, SuperSplat Viewer 1.37.0, splat-transform 3.9.0–3.10.1, Python with NumPy and SciPy).

5.2. Reconstruction quality

On `vr_room` (Table 4), DN-Splatter's priors cost 0.5 dB of PSNR on held-out frames of the training walk but gain 0.26 dB on the separate walk, and its mesh is 3.3 times closer to the laser scan (1.76 vs 5.85 cm, F-score 0.93 vs 0.64). Since walk collision is built from geometry, we kept DN-Splatter. Neither run collapsed (Figure 3, right). Our own room reached 22.2 dB on its held-out frames, a different room and capturer, so not directly comparable. Training took 87 minutes (DN-Splatter) and 79 minutes (plain) on `vr_room`, and 99 minutes on our room including one crash and automatic resume; our peak GPU memory was about 5.2–5.3 GB, and the final splats held 0.91 M and 0.75 M Gaussians.

Table 4 Reconstruction quality after 30,000 steps (GPU machine). Depth is compared with the phone's LiDAR, on which DN-Splatter trains, so that column favours it; mesh metrics are against the laser scan.

Run	**Within**			**With**			**Depth**	**Mesh acc.**	**F-score**
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	abs. rel.	(cm)	@5 cm
`vr_room`, plain splats	**26.84**	**0.875**	**0.124**	23.24	**0.818**	0.192	0.056	5.85	0.638
`vr_room`, DN-Splatter	26.32	0.862	0.143	**23.50**	0.812	0.192	**0.016**	**1.76**	**0.926**
Own room, DN-Splatter	22.18	0.853	0.170	–	–	–	0.017	–	–

Figure 5 shows renders next to the photographs. Near the training walk, signs, pipes and the door are sharp; from the other walk, the blanket and

the wall seen close up are streaky. In our room the patterned mattress is sharp but the plain walls and ceiling are smeared. Plain walls remain the weak point even with depth and normal priors.



Figure 5 Renders on held-out frames (GPU machine). (a) A held-out frame of the `vr_room` training walk and (b) a frame of its separate walk, render left; (c) our own room, render right.

5.3. Walkability

345 Both rooms were walked in the SuperSplat Viewer on the laptop (Table 5). In `vr_room` the walker starts inside the room at eye height, walls stop the walker at the visible surface and the floor holds; our own room was judged good by Aswin, without recording each check. In both screen recordings the overlay showed a frame time of 10.0 ms at every sampled moment while the

GPU time varied, which suggests the display, not rendering, set the frame rate. Close to a million Gaussians therefore needed no pruning, unlike the 400–500k limit of v1's browser viewer.

Table 5 Walk tests (laptop, Chrome). Times are read from the viewer's overlay in the screen recordings at regular samples.

Room	Gaussians	Spawn inside	Walls stop, floor holds	Frame	GPU
`vr_room`	908k	yes	yes	10.0 ms	2.5–8.3 ms
Own room	746k	judged good (not itemised)		10.0 ms	2.9–7.1 ms

5.4. Walk-collision accuracy

Table 6 and Figure 6 give the collision study. The default collision from the splat (S) has gaps at body height, along one wall and at furniture, so the walkable area leaks out of the room (11.3 m^2 against the scan's 6.5 m^2): IoU 0.58, with 17.8% of the walls the phone saw missing. Collision from DN-Splatter's mesh (M) follows the scan closely: IoU 0.81, 1.7% missed, and a 95th-percentile offset of 10 instead of 20 cm, but with 0.04 m^2 of phantom obstacles (16 cells of 5 cm) where S has none. M therefore passes P1, P3 and P4 (the median offset ties at 0 at this resolution) and fails P2, so the all-four rule we set is not met. Raw phone LiDAR (L) is unusable for collision (IoU 0.06, 4.3 m^2 of phantom obstacles from noise and stray points), so the training step that filters the depth matters. Polycam's mesh (P), aligned in its best case, is almost as good as M but with more phantom obstacles.

The difference was not felt in practice: walking both versions freely, Aswin noticed no difference between S and M. The gaps lie at furniture and wall

Table 6 Walk-collision accuracy against the laser scan of `vr_room` (laptop). All sources use the same splat-transform pipeline; measured on a 5 cm grid between 0.2 and 1.8 m above the floor.

Source	IoU ↑	Phantom ↓	Missed ↓	Offset med./p95	Walkable
S: splat (default)	0.577	**0.00** m^2	17.8%	0 / 20 cm	11.3 m^2
M: DN-Splatter mesh	**0.809**	0.04 m^2	**1.7%**	0 / **10** cm	7.6 m^2
P: Polycam mesh	0.794	0.22 m^2	1.8%	0 / 10 cm	6.9 m^2
L: raw phone LiDAR	0.059	4.27 m^2	47.0%	5 / 41 cm	11.4 m^2
GT: laser scan	1	0	0%	0 / 0 cm	6.5 m^2

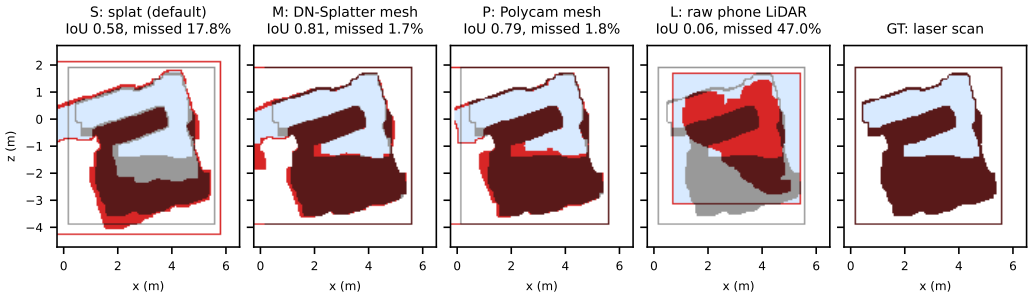


Figure 6 Floor plans of the `vr_room` walk collision at body height. Red: the source's collision; grey: the laser scan's; dark red: both; light blue: walkable from the spawn. The outer rectangles close each collision volume's bounding box.

stretches that a free walk did not reach. The study has clear limits: one room (the only one of ours with a laser scan); the reference is the scan's own collision through the same 5 cm pipeline; the 5 cm cells make the median offset coarse; and S uses the real Gaussians while M, L, P and GT were converted into 2 cm splats, which may itself contribute to the difference.

5.5. Negative results: capture check and tracking repair

Capture check. Its verdicts matched what we knew: `cap3` was flagged as broken tracking (Figure 7, left), and `cap1`, `cap2` and `vr_room` were asked to recapture some areas but not flagged as broken; it ran in 18–52 s per room

on the laptop. Its pre-agreed validation, comparing labels with the trained model's error on held-out frames, failed both criteria (Table 7). Under-covered patches had only slightly higher error than OK patches, with over-
380 lapping intervals, and textureless patches had *lower* error. Coverage predicted the worst 20% of patches no better than chance (AUC 0.53–0.54), and texture added 0.046 AUC, short of the 0.05 required. A likely reason, found afterwards, is that a per-pixel colour error rewards smooth renders of plain walls: with 1−SSIM as the error (not pre-agreed), texture separated
385 bad patches with an AUC of 0.73–0.82, in the direction "more detail, more error". The check is kept as a fallback, not claimed as a result.

Table 7 Capture-check validation (laptop): mean held-out patch error (absolute colour error, 0–1) with bootstrap 95% intervals. The criterion required under-covered and textureless patches to exceed OK patches with non-overlapping intervals in the test room.

Room	**OK**	**Under-covered**	**Textureless**
`vr_room` (calibration)	0.033 [0.029, 0.036]	0.036 [0.033, 0.039]	0.016 [0.010, 0.022]
`cap2` (test)	0.056 [0.049, 0.065]	0.063 [0.053, 0.074]	0.034 [0.020, 0.057]

Tracking repair. The second attempt registered `cap3`'s separate sweeps to each other before training. The registration worked (fitness 0.87, turn −0.9°, tilt 0.6°), but there was nothing to repair: before any repair the first
390 sweep's depth already lay within 2.4 cm of the main sweep with Polycam's corrected poses (2.7 cm with ARKit's), close to the main sweep's 1.9 cm agreement with itself. ARKit had relocalised, and only seven transition frames (118–124) carried the 2 m jumps. The pre-agreed premise was false, and the added surface (4.7%) fell below the required 5%.

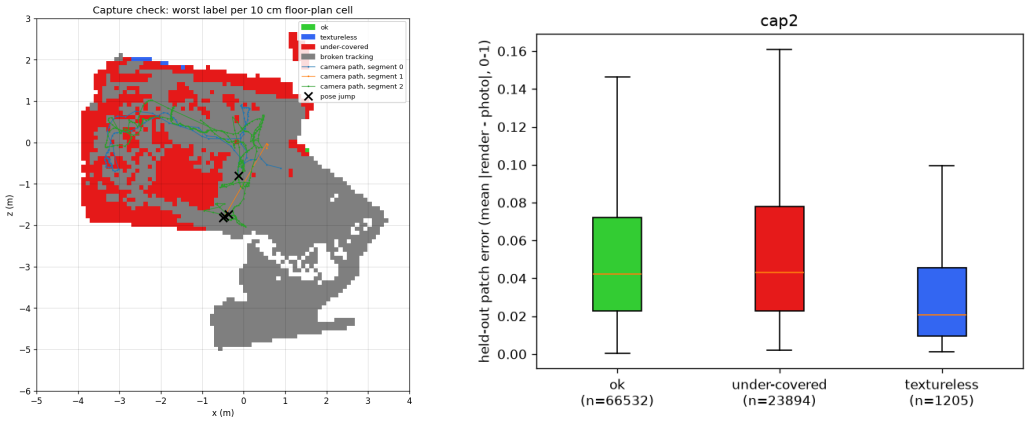


Figure 7 Capture check. Left: top-down map of `cap3`, worst label per 10 cm cell, with the camera path coloured by tracking segment and pose jumps marked; grey areas were seen only by frames with broken tracking. Right: held-out patch error by label in the test room; textureless patches have lower error, against the check's claim.

5.6. Comparison with existing implementations

Table 8 compares existing ways to obtain the model or its walk collision with our pipeline, all run and measured by us on `vr_room`. Plain splats render the training walk best but give the least accurate geometry; the viewer's default collision from the splat leaks out of the room; raw phone LiDAR cannot be used for collision; and Polycam's mesh is almost as accurate as DN-Splatter's (1.86 vs 1.76 cm, F-score 0.920 vs 0.926) and gives collision almost as good, with more phantom obstacles, even though its alignment was fitted to the same laser scan, which favours it (we did not evaluate the app's other outputs). Polycam's mesh accuracy was measured on a Kaggle T4, because the GPU machine had lost GPU access and the protocol renders depth maps on the GPU; the environment used the same library versions.

Our combination, DN-Splatter's splat for viewing and its mesh for collision, has the most accurate geometry and the best collision on every measure except phantom obstacles, but its margin over Polycam's mesh is small.

Table 8 Comparison of existing implementations with ours on `vr_room` (our measurements; "–": not applicable or not measured). Mesh metrics follow DN-Splatter's MuSH-Room protocol (GPU machine; Polycam's on a Kaggle T4).

Method (implementation)	PSNR within / with	Mesh acc.	F@5 cm	Walk IoU	Phantom	Missed
Plain 3DGS splats [4] (nerfstudio, priors off)	**26.84** / 23.24 dB	5.85 cm	0.638	–	–	–
SuperSplat default: collision from the splat [11]	same splat as ours	–	–	0.577	**0.00** m^2	17.8%
Raw phone LiDAR, no reconstruction	–	–	–	0.059	4.27 m^2	47.0%
Polycam app mesh [41]	–	1.86 cm	0.920	0.794	0.22 m^2	1.8%
Ours: DN-Splatter splat + mesh collision	26.32 / **23.50** dB	**1.76 cm**	**0.926**	**0.809**	0.04 m^2	**1.7%**

6. Conclusion and future scope

RoomRecon turns an iPhone capture of a room into a photoreal Gaussian splat that can be walked through with collision, using maintained tools for almost every stage and our own code for the glue and the evaluation. Taking poses and depth from the phone, instead of estimating poses from images as in our first attempt, removed the failure on plain walls, and DN-Splatter's priors made the geometry 3.3 times more accurate than plain splats (1.76 vs 5.85 cm from the laser scan) for 0.5 dB less PSNR near the training walk. A public room and our own room were both walkable on a 6 GB laptop GPU. Our main measurement shows that walk collision built from DN-Splatter's mesh matches a laser scan far better than the default collision built from the splat: the walkable-area IoU rose from 0.58 to 0.81 and missed walls fell from

17.8% to 1.7%, at the cost of 0.04 m^2 of phantom obstacles, which failed one of our four pre-agreed criteria. Polycam's mesh of the same capture performed almost as well, both as collision (0.79, 1.8%) and as geometry (1.86 cm from the scan), so the main gain is over the default splat collision, not over existing apps. The gap was not felt in a free walk and the study covers one room, so we present it as a measured difference, not a general law. Two other novelty attempts, a capture check and a tracking repair, failed the tests we set in advance; we report them because they show what did not work and why.

Two directions follow from this work:

1. **Repeat the collision study on more rooms.** MuSHRoom provides reference meshes for all ten of its rooms, which would show whether the advantage of mesh-based collision holds beyond `vr_room` and whether the mesh should become the default collision source.
2. **Poses from images for phones without LiDAR.** Feed-forward models such as VGGT [26] and MASt3R-SfM [25] could be tested on the plain-walled bedroom that defeated our v1.

Acknowledgements

We thank IIIT Kottayam for access to its GPU server, and Kaggle for free GPU sessions. We used the MuSHRoom dataset [13] (CC BY 4.0) and, in v1, Apple's ARKitScenes [38]. Our pipeline is built on open-source work by others: nerfstudio [12], DN-Splatter [5, 45], gsplat [14], Omnidata [42], Open3D [43], PlayCanvas' splat-transform [11] and SuperSplat Viewer [10],

and pixi [44]; captures were made with Polycam [41]. In v1 we also used COLMAP [20], hloc [23] with SuperPoint [21] and LightGlue [22], ORB-SLAM3 [24], GaussianSplats3D [39] and NeRFCapture [40].

References

[1] Z. Chen, H. Bao, H. Chang, A. Yu, R. Choi, K. McClennen, G. Huh, K. Yang, R.-Z. Qiu, Y. Ravan, J. J. Leonard, X. Wang, P. Isola, G. Yang, Y. Wang, The Neverwhere visual parkour benchmark suite, arXiv preprint arXiv:2609.16443Accepted to IROS 2026.

[2] J. Liu, Y. Duan, J. Zhang, M. Li, S. Wang, Z. Zhang, H. Wang, NavGSim: High-fidelity Gaussian splatting simulator for large-scale navigation, arXiv preprint arXiv:2603.15186.

[3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, R. Ng, NeRF: Representing scenes as neural radiance fields for view synthesis, in: Proceedings of the European Conference on Computer Vision (ECCV), 2020.

[4] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis, 3D Gaussian splatting for real-time radiance field rendering, ACM Transactions on Graphics 42 (4).

[5] M. Turkulainen, X. Ren, I. Melekhov, O. Seiskari, E. Rahtu, J. Kannala, DN-Splatter: Depth and normal priors for Gaussian splatting and

meshing, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2025.

[6] X. Ren, M. Turkulainen, J. Wang, O. Seiskari, I. Melekhov, J. Kannala, E. Rahtu, AGS-Mesh: Adaptive Gaussian splatting and meshing with geometric priors for indoor room reconstruction using smartphones, in: Proceedings of the International Conference on 3D Vision (3DV), 2025.

[7] C. Ruan, Y. Wang, T. Guan, B. Zhang, L. Ju, IndoorGS: Geometric cues guided Gaussian splatting for indoor scene reconstruction, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025.

[8] X. Jin, R. Jin, B. Li, D. Zou, W. Yu, PlanarGS: High-fidelity indoor 3D Gaussian splatting guided by vision-language planar priors, in: Advances in Neural Information Processing Systems (NeurIPS), 2025.

[9] PlayCanvas, New in SuperSplat: Walk mode, streamed LOD and easy upload, `https://blog.playcanvas.com/new-in-supersplat-walk-mode-streamed-lod-and-easy-upload`, blog post, accessed 2026-10-09.

[10] PlayCanvas, SuperSplat viewer, `https://github.com/playcanvas/supersplat-viewer`, version 1.37.0, used 2026-10.

[11] PlayCanvas, SplatTransform: Gaussian splat conversion and colli-

sion tool, https://github.com/playcanvas/splat-transform, versions 3.9.0–3.10.1, used 2026-10.

[12] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, J. Kerr, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. McAllister, A. Kanazawa, Nerfstudio: A modular framework for neural radiance field development, in: ACM SIGGRAPH 2023 Conference Proceedings, 2023, pp. 1–12. doi:10.1145/3588432.3591516.

[13] X. Ren, W. Wang, D. Cai, T. Tuominen, J. Kannala, E. Rahtu, MuSHRoom: Multi-sensor hybrid room dataset for joint 3D reconstruction and novel view synthesis, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024, pp. 4508–4517.

[14] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik, A. Kanazawa, gsplat: An open-source library for Gaussian splatting, Journal of Machine Learning Research 26 (34) (2025) 1–17.

[15] B. Huang, Z. Yu, A. Chen, A. Geiger, S. Gao, 2D Gaussian splatting for geometrically accurate radiance fields, in: ACM SIGGRAPH 2024 Conference Papers, 2024. doi:10.1145/3641519.3657428.

[16] S. Kheradmand, D. Rebain, G. Sharma, W. Sun, Y.-C. Tseng, H. Isack, A. Kar, A. Tagliasacchi, K. M. Yi, 3D Gaussian splatting as Markov

chain Monte Carlo, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024.

[17] H. Xiang, X. Li, K. Cheng, X. Lai, W. Zhang, Z. Liao, L. Zeng, X. Liu, GaussianRoom: Improving 3D Gaussian splatting with SDF guidance and monocular cues for indoor scene reconstruction, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2025.

[18] S. Han, J. Jang, H. Kim, J. Surh, J. Kwak, H. Ha, K. Joo, LighthouseGS: Indoor structure-aware 3D Gaussian splatting for panorama-style mobile captures, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2026.

[19] W. Guo, G. Fang, S. Yang, B. Wang, PocketGS: On-device training of 3D Gaussian splatting for high perceptual modeling, arXiv preprint arXiv:2601.17354.

[20] J. L. Schönberger, J.-M. Frahm, Structure-from-motion revisited, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 4104–4113.

[21] D. DeTone, T. Malisiewicz, A. Rabinovich, SuperPoint: Self-supervised interest point detection and description, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Deep Learning for Visual SLAM, 2018.

[22] P. Lindenberger, P.-E. Sarlin, M. Pollefeys, LightGlue: Local feature matching at light speed, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 17627–17638.

[23] P.-E. Sarlin, C. Cadena, R. Siegwart, M. Dymczyk, From coarse to fine: Robust hierarchical localization at large scale, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019.

[24] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, J. D. Tardós, ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multimap SLAM, IEEE Transactions on Robotics 37 (6) (2021) 1874–1890.

[25] B. Duisterhof, L. Žust, P. Weinzaepfel, V. Leroy, Y. Cabon, J. Revaud, MASt3R-SfM: A fully-integrated solution for unconstrained structure-from-motion, in: Proceedings of the International Conference on 3D Vision (3DV), 2025.

[26] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, D. Novotny, VGGT: Visual geometry grounded transformer, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025.

[27] T. Chen, O. Shorinwa, J. Bruno, A. Swann, J. Yu, W. Zeng, K. Nagami, P. Dames, M. Schwager, Splat-Nav: Safe real-time robot naviga-

tion in Gaussian splatting maps, IEEE Transactions on RoboticsArXiv:2403.02751.

[28] A. Yu, G. Yang, R. Choi, Y. Ravan, J. Leonard, P. Isola, Learning visual parkour from generated images, in: Proceedings of the 8th Conference on Robot Learning (CoRL), Vol. 270 of Proceedings of Machine Learning Research, 2024, pp. 2500–2516.

[29] G. Kopanas, G. Drettakis, Improving NeRF quality by progressive camera placement for free-viewpoint navigation, in: Vision, Modeling, and Visualization (VMV), The Eurographics Association, 2023, pp. 11–20. doi:10.2312/vmv.20231222.

[30] W. Jiang, B. Lei, K. Daniilidis, FisherRF: Active view selection and mapping with radiance fields using Fisher information, in: Proceedings of the European Conference on Computer Vision (ECCV), 2024. doi:10.1007/978-3-031-72624-8_24.

[31] W. Xiao, R. Santa Cruz, D. Ahmedt-Aristizabal, O. Salvado, C. Fookes, L. Lebrat, NeRF director: Revisiting view selection in neural volume rendering, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.

[32] T. Chen, A. Dai, M. Adang, G. Gao, M. Schwager, Coverage optimization for camera view selection, arXiv preprint arXiv:2604.05259.

[33] A. Mollaei Khass, N. Motee, AGILE-GS: Anchor-guided fast next-best-view selection for active 3D Gaussian splatting, arXiv preprint arXiv:2609.34176.

[34] Y. Zhang, L. Bai, M. Yu, L. Wei, S. Wan, Y. Zhang, An object-centered data acquisition method for 3D Gaussian splatting using mobile phones, arXiv preprint arXiv:2604.19216.

[35] A. Yasunaga, H. Saito, S. Mori, User-in-the-loop view sampling with error peaking visualization, in: Proceedings of the IEEE International Conference on Image Processing (ICIP), 2025, arXiv:2506.21009.

[36] H. Chang, J. B. Choi, C. M. Yeum, 3D reconstruction by looking: Instantaneous blind spot detector for indoor SLAM through mixed reality, Advanced Engineering Informatics (2026) 104065ArXiv:2411.12514.

[37] T. Chen, J. Jin, S. Cai, Z. Li, W. Lin, MUGSQA: Novel multi-uncertainty-based Gaussian splatting quality assessment method, dataset, and benchmarks, in: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2026, arXiv:2511.06830.

[38] G. Baruch, Z. Chen, A. Dehghan, T. Dimry, Y. Feigin, P. Fu, T. Gebauer, B. Joffe, D. Kurz, A. Schwartz, E. Shulman, ARKitScenes: A diverse real-world dataset for 3D indoor scene understanding using

mobile RGB-D data, in: Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1), 2021.

[39] M. Kellogg, GaussianSplats3D: Three.js-based Gaussian splat viewer, https://github.com/mkkellogg/GaussianSplats3D, version 0.4.7, used in v1 (2026-09).

[40] J. Abou-Chakra, NeRFCapture, https://github.com/jc211/NeRFCapture, iOS app (2023).

[41] Polycam Inc., Polycam: 3D scanning app, https://poly.cam, iOS app, raw-data export used 2026-10.

[42] A. Eftekhar, A. Sax, R. Bachmann, J. Malik, A. Zamir, Omnidata: A scalable pipeline for making multi-task mid-level vision datasets from 3D scans, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021.

[43] Q.-Y. Zhou, J. Park, V. Koltun, Open3D: A modern library for 3D data processing, arXiv preprint arXiv:1801.09847.

[44] prefix.dev, pixi: package management made easy, https://pixi.sh, version 0.81.0.

[45] M. Turkulainen, et al., DN-Splatter source code, https://github.com/maturk/dn-splatter, commit 97588b4, used 2026-10.